

Dobbertin's Theorem 1 — Equation (1) and the step (1) $\Rightarrow$ (2)

A paper-to-Lean 4 map, in the topological order of the formalisation
from the Mathlib foundations to the headline results

Formalisation library `Dobbertin1999MVP.Equation1.*`

Abstract

This document places, *side by side*, the relevant passages of H. Dobbertin, *Kasami Power Functions, Permutation Polynomials and Cyclic Difference Sets* (1999, pp. 133–138) and their Lean 4 formalisation in the `Dobbertin1999MVP.Equation1.*` library. The material is ordered *exactly as the Lean library is built* — i.e. along a topological sort of the module dependency DAG, from the Mathlib foundations at the bottom up to the headline results `theorem_1`, `eqn2_of_eqn1` (the step (1) $\Rightarrow$ (2)) and the two case analyses. Each layer is accompanied by TikZ dependency diagrams and `tikz-cd` commutative diagrams visualising how the paper's argument is threaded through the Lean declarations. Every declaration quoted here is **sorry-free** in the sources and, as recorded in the project's architecture report, rests only on the standard axioms `propext`, `Classical.choice`, `Quot.sound`.

Contents

1	How to read this document	2
2	The dependency DAG (Mathlib $\rightarrow$ headlines)	2
3	Layer 1a — Defs : the pure definitions	4
4	Layer 1b — FiniteFieldPrereqs : the finite-field engine	4
5	Layer 2a — Setup : the elementary opening of the proof	5
6	Layer 2b — Theorem5 : the trace-free permutation criterion	6
7	Layer 2c — Theorem8Trace : the two trace identities	7
8	Layer 3a — Theorem8C1 : $\text{Tr}(x) \in \{0, 1\}$ and gmap	7
9	Layer 3b — Q1General : the $\alpha = 1$ criterion	7
10	Layer 4 — Equation1 : the headline thread	8
10.1	Bridges: <code>qKasami</code> $\leftrightarrow$ <code>qeps</code> / <code>gmap</code>	8
10.2	Theorem 1	8
10.3	The step (1) $\Rightarrow$ (2)	9
10.4	The two cases	9
11	The two documented corrections	10
12	Alternative foundations and proof paths (summary)	10

1 How to read this document

The library is a self-contained extract carrying exactly the material needed, end to end from **Mathlib**, for **equation (1)** of Theorem 1 and its first substantive step $(1) \Rightarrow (2)$. Equation (1) is the equation $q_\alpha(x) = c$ cleared of denominators,

$$c x^{2^k+1} = \sum_{i=1}^{k'} x^{2^{ik}} + \alpha \operatorname{Tr}(x). \quad (1)$$

The exposition follows the *compile order* of the library. Concretely, a Lean file may only refer to declarations that appear earlier in the dependency DAG, so reading the modules in topological order is the same as reading the mathematics “from the foundations up”. We use two visually distinct blocks:

- a blue-barred *paper quote* block for passages of Dobbertin (1999); and
- a boxed Lean listing for the corresponding formalisation.

The global correspondence is summarised in Table 1.

Table 1: Paper object $\leftrightarrow$ Lean declaration (headline thread).

Paper object	Lean (Dobbertin1999.Paper)	status
Trace $\operatorname{Tr}(x) = \sum_{i < n} x^{2^i}$	<code>def Tr</code>	def
Generalized Kasami poly. q_α	<code>def qKasami</code>	def
Equation (1)	<code>def eqn1</code>	def
$\ell(x) = c^{2^k} x^{2^{2k}} + x^{2^k} + cx + 1$	<code>def ell</code>	def
$(2^k - 1)^{-1} \pmod{2^n - 1}$ exists	<code>inv_mod_exists</code>	✓
“only if” at $q_\alpha(1)$	<code>qKasami_one_eq_zero_iff</code>	✓
Theorem 1 (permutation criterion)	<code>theorem_1</code>	✓
Step (1) $\Rightarrow$ (2)	<code>eqn2_of_eqn1</code>	✓
Case 1 (unique root of ℓ)	<code>theorem_1_case1</code>	✓
Case 2 factorisation $\ell = Q^{2^k} + fQ$	<code>ell_eq_0</code>	✓
Case 2 (unique nonzero solution of (1))	<code>theorem_1_case2</code>	✓

2 The dependency DAG (Mathlib $\rightarrow$ headlines)

The library is eight modules in four layers. Figure 1 is the full module dependency graph; arrows are **import/uses** edges and point in the direction of dependence (a module points to the modules it is built on). Reading the figure *bottom-to-top* is the topological order used by the rest of this document.

The concrete Mathlib surface everything bottoms out on is small and stable; Figure 2 maps those foundation lemmas to the modules that consume them.

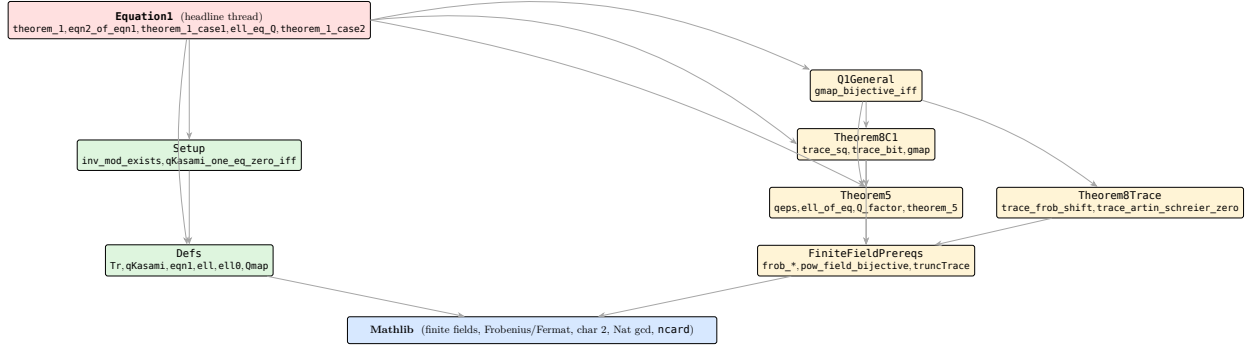


Figure 1: Module dependency DAG. Colour code: blue = **Mathlib**; green = pure definitions / elementary opening; amber = finite-field engine; red = the equation-(1) headline thread. Read bottom-to-top.

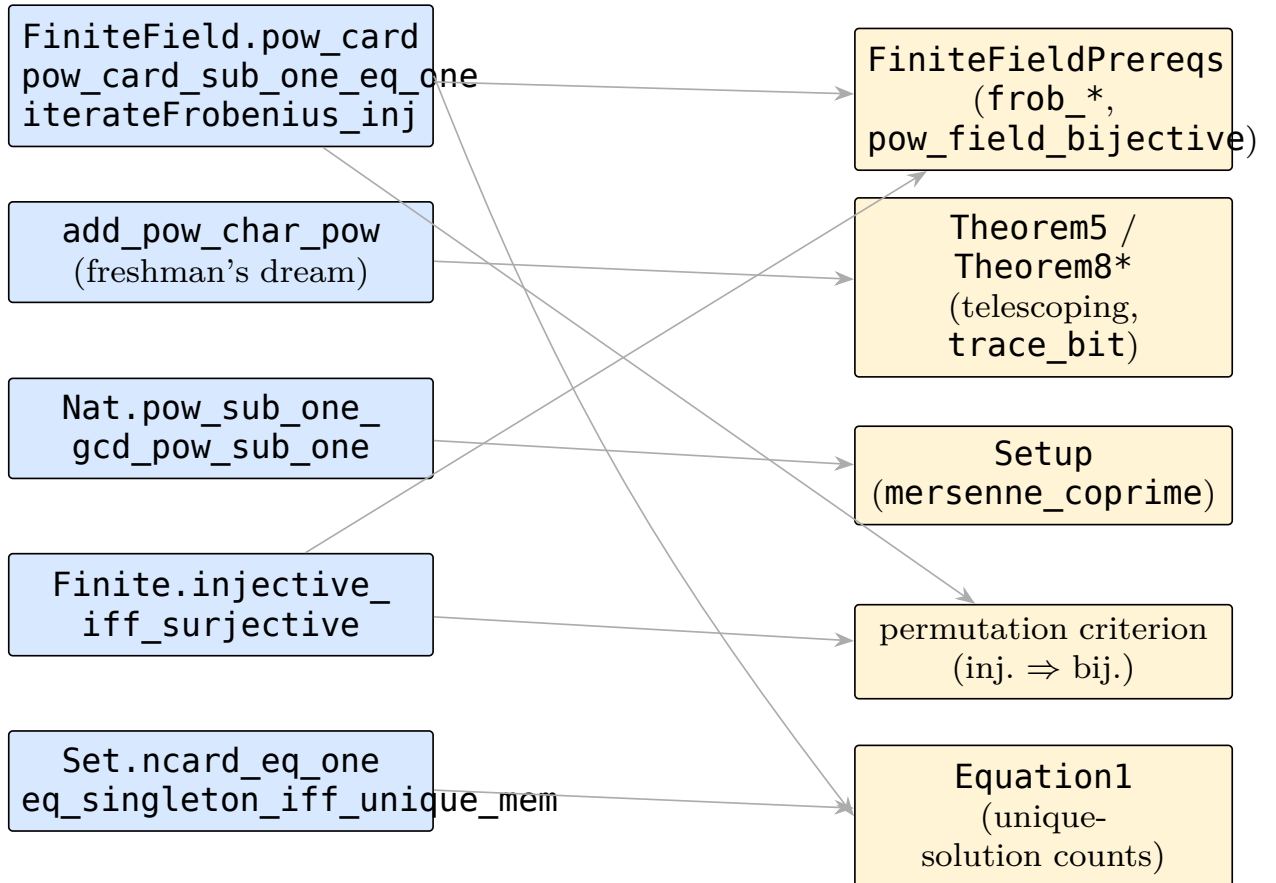


Figure 2: The non-trivial Mathlib foundations (left) and the library layers that consume them (right). Elementary **Nat** arithmetic and general automation (**ring**, **grind**, **decide**, **simp**, **omega**) sit underneath all of these.

3 Layer 1a — Defs: the pure definitions

The bottom of the DAG is a single Mathlib-only file of definitions. It fixes the ambient field and transcribes the paper’s objects.

Dobbertin (1999) — p. 134, standing assumptions and q_α

We always assume $\gcd(k, n) = 1$, $k < n$, $k' \equiv 1/k \pmod{n}$. For $\alpha = 0, 1$ we define the polynomials

$$q_\alpha(z) = \frac{\sum_{i=1}^{k'} z^{2^{ik}} + \alpha \operatorname{Tr}(z)}{z^{2^k+1}},$$

where Tr denotes the trace function from $\mathbb{F}_{2^n}$ onto $\mathbb{F}_2$. (To be formally more precise, we get a polynomial q_α for each $L = \mathbb{F}_{2^n}$ if we substitute $1/z^{2^k+1}$ by $z^{(2^n-1)-(2^k+1)}$.) We call q_α a generalized Kasami polynomial.

The library realises $L = \mathbb{F}_{2^n}$ abstractly as any finite field of characteristic 2, and the “ $0/0 = 0$ ” convention as the honest power $z^{(2^n-1)-(2^k+1)}$.

```
variable {L : Type*} [Field L] [Fintype L] [CharP L 2]

/-- The absolute trace `Tr(x) = \sum_{i<n} x^{2^i}`. -/
def Tr (n : ℕ) (x : L) : L := ∑ i ∈ Finset.range n, x ^ (2 ^ i)

/-- The generalized Kasami polynomial `αq_` (with `0/0 = 0`). -/
def qKasami (n k k' : ℕ) α( : ℕ) (z : L) : L :=
  ∑( ( i ∈ Finset.Icc 1 k', z ^ (2 ^ (i * k))) + α( : L) * Tr n z
    * z ^ (2 ^ n - 1 - (2 ^ k + 1))

/-- Equation (1): `αq_(x) = c` cleared of denominators. -/
def eqn1 (n k k' : ℕ) α( : ℕ) (c x : L) : Prop :=
  c * x ^ (2 ^ k + 1) = ∑( i ∈ Finset.Icc 1 k', x ^ (2 ^ (i * k))) + α( : L) * Tr n
  x
```

The auxiliary polynomials of equation (2) are defined here too:

```
/-- `ell(x) = c^{2^k} · x^{2^{2k}} + x^{2^k} + c · x + 1` (equation (2)). -/
def ell (k : ℕ) (c x : L) : L :=
  c ^ (2 ^ k) * x ^ (2 ^ (2 * k)) + x ^ (2 ^ k) + c * x + 1
/-- Homogeneous part `ell0(x) = ell(x) + 1`. -/
def ell0 (k : ℕ) (c x : L) : L := ell (L := L) k c x + 1
/-- `Q(x) = c · x^{2^k} + y^2 · x + y` (Case 2). -/
def Qmap (k : ℕ) (c y x : L) : L := c * x ^ (2 ^ k) + y ^ 2 * x + y
```

4 Layer 1b — FiniteFieldPrereqs: the finite-field engine

Parallel to **Defs**, and also resting only on Mathlib, is the finite-field engine. It packages the two devices the whole proof runs on: the *Frobenius map as a bijection*, and the *truncated trace* $L(x) = \sum_{i<m} x^{2^i}$ with its additivity and Artin–Schreier telescoping.

```
lemma frob_cycle (x : F) : x ^ Fintype.card F = x := FiniteField.pow_card x

lemma frob_bijective (r : ℕ) : Function.Bijective (fun x : F => x ^ (p ^ r)) :=
  (iterateFrobenius_inj F p r,
   (Finite.injective_iff_surjective).mp (iterateFrobenius_inj F p r))

/-- `x ↦ a·x` is a bijection when `gcd(|F|-1, a) = 1`. -/
lemma pow_field_bijective {a : ℕ} (ha : Nat.Coprime (Fintype.card F - 1) a)
  (ha_pos : 0 < a) : Function.Bijective (fun x : F => x ^ a)
```

```

/-- The truncated trace `L(x) = \sum_{i < m} x^{2^i}`. -/
def truncTrace (m : ℕ) (x : F) : F := \sum i \in Finset.range m, x ^ (2 ^ i)

/-- Additivity of `L` (freshman's dream, char 2). -/
lemma truncTrace_add (m : ℕ) (x y : F) :
  truncTrace m (x + y) = truncTrace m x + truncTrace m y

/-- Artin-Schreier telescoping: `L(x)^2 + L(x) = x^{2^m} + x`. -/
lemma truncTrace_sq_add_self (m : ℕ) (x : F) :
  truncTrace m x ^ 2 + truncTrace m x = x ^ (2 ^ m) + x

```

The additivity of L is exactly the “freshman’s dream” $(x + y)^{2^i} = x^{2^i} + y^{2^i}$ summed over i , and the telescoping identity is the algebraic heart of every trace computation downstream. Diagrammatically:

$$\begin{array}{ccc}
 F \times F & \xrightarrow{+} & F \\
 L \times L \downarrow & & \downarrow L \\
 F \times F & \xrightarrow{+} & F
 \end{array}
 \qquad
 \begin{array}{ccc}
 x & \xrightarrow{L} & L(x) \\
 (\cdot)^2 + \cdot \downarrow & & \downarrow (\cdot)^2 + \cdot \\
 x^{2^m} + x & \xlongequal{\quad} & L(x)^2 + L(x)
 \end{array}$$

Figure 3: Left: additivity of the truncated trace (`truncTrace_add`). Right: the Artin–Schreier telescoping (`truncTrace_sq_add_self`).

5 Layer 2a — **Setup:** the elementary opening of the proof

This module is the engine-free opening of Theorem 1’s proof — everything *before* equations (1)/(2).

Dobbertin (1999) — p. 135, opening of the proof of Theorem 1

Proof. First note that $(2^k - 1)^{-1} \pmod{2^n - 1}$ exists, since $\gcd(k, n) = 1$. We have $q_\alpha(0) = 0$, using the convention “ $0/0 = 0$ ”. To verify the “only if” part, observe that $k' + \alpha n \equiv 0 \pmod{2}$ is equivalent to $q_\alpha(1) = k' \cdot 1 + \alpha \cdot \text{Tr}(1) = 0$.

Each sentence maps to one declaration. The existence of the modular inverse reduces to the Mersenne-gcd fact $\gcd(2^k - 1, 2^n - 1) = 2^{\gcd(k, n)} - 1$:

```

theorem mersenne_coprime (h : Nat.Coprime k n) :
  Nat.Coprime (2 ^ k - 1) (2 ^ n - 1) := by

  unfold Nat.Coprime at *; rw [Nat.pow_sub_one_gcd_pow_sub_one]; simp [h]

theorem inv_mod_exists (h : Nat.Coprime k n) (hn : 1 < 2 ^ n - 1) :
  \exists b, (2 ^ k - 1) * b % (2 ^ n - 1) = 1

theorem qKasami_zero \alpha ( : ℕ) : qKasami (L := L) n k k' \alpha 0 = 0

theorem qKasami_one \alpha ( : ℕ) :
  qKasami (L := L) n k k' \alpha 1 = ((k' + \alpha * n : ℕ) : L)

/-- The "only if" part at the value \alpha q_1(1). -/
theorem qKasami_one_eq_zero_iff \alpha ( : ℕ) :
  qKasami (L := L) n k k' \alpha 1 = 0 \leftrightarrow (k' + \alpha * n) % 2 = 0

```

6 Layer 2b — Theorem5: the trace-free permutation criterion

This is the $\alpha = 0$ backbone and the source of the three reduction lemmas reused by the equation-(1) thread. Dobbertin’s Theorem 5 treats the *trace-free* polynomial $q^{(\varepsilon)}$, and its proof *is* the root-count argument that Theorem 1 quotes.

Dobbertin (1999) — p. 135, the reduction to the linearised equation

To prove the “if” part assume $k' + \alpha n \equiv 1 \pmod{2}$. We shall show that for each fixed $c \in L$, the equation

$$c x^{2^k+1} = \sum_{i=1}^{k'} x^{2^{ik}} + \alpha \text{Tr}(x) \quad (1)$$

has at most one solution in L . Adding the 2^k -th power of this equation to itself we get ... and consequently

$$\ell(x) = c^{2^k} x^{2^{2k}} + x^{2^k} + cx + 1 = 0. \quad (2)$$

The module first sets up the step- 2^k partial trace $P(x) = \sum_{j < k'} x^{2^{jk}}$ and the numerator sum $S(x) = \sum_{i=1}^{k'} x^{2^{ik}}$, with $S = P^{2^k}$ and the telescoping $P^{2^k} + P = x^2 + x$ (via `pTrace_telescope`). The (1) $\Rightarrow$ (2) reduction is then packaged as `ell_of_eq`:

```
noncomputable def pTrace (k k' : ℕ) (x : F) : F := ∑ j ∈ Finset.range k', x ^ (2 ^ (j * k))
noncomputable def sTrace (k k' : ℕ) (x : F) : F := ∑ i ∈ Finset.Icc 1 k', x ^ (2 ^ (i * k))
noncomputable def qeps (n k k' : ℕ) ε ( : F) (x : F) : F :=
  (sTrace k k' x + ε) * x ^ (2 ^ n - 1 - (2 ^ k + 1))
/-- (1) => (2): on units, `qε^{()}(x) = c` gives `ell(x) = 0`. -/
lemma ell_of_eq (hn : Fintype.card F = 2 ^ n) (hkk' : k * k' % n = 1)
  (εh : ε = 0 ∨ ε = 1) (hx : x ≠ 0) (hex : sTrace k k' x + ε = c * x ^ (2 ^ k + 1))
  ) :
  c ^ (2 ^ k) * x ^ (2 ^ (2 * k)) + x ^ (2 ^ k) + c * x + 1 = 0
```

The two case lemmas of Theorem 1 also live here, in their trace-free form:

```
/-- Case 1: a nonzero root of ell0 forces `c` into the image of γ ↦ γ^{2^kγ+1}. -/
lemma ell0_root_imp_image (hn : Fintype.card F = 2 ^ n) (hk : 0 < k) (hkn : k < n)
  (hc : c ≠ 0) (ht : t ≠ 0)
  (h0 : c ^ (2 ^ k) * t ^ (2 ^ (2 * k)) + t ^ (2 ^ k) + c * t = 0) :
  ∃ γ : F, c = γ ^ (2 ^ k + 1) + γ
/-- Case 2: the factorisation `ell = Q^{2^k} + f·Q` with `f = γ^{2^kγ-1}+1`. -/
lemma Q_factor (γh : γ ≠ 0) (hcd : c = γ ^ (2 ^ k + 1) + γ) (t : F) :
  c ^ (2 ^ k) * t ^ (2 ^ (2 * k)) + t ^ (2 ^ k) + c * t + 1
  = (qPoly γ c k t) ^ (2 ^ k) + γ ( ^ (2 ^ k - 1) + γ^{-1}) * (qPoly γ c k t)
```

Culminating in the trace-free criterion:

```
/-- Theorem 5: `qε^{()}` is a permutation of `F` iff ε ≡ k'+1 (mod 2). -/
theorem theorem_5 (hn : Fintype.card F = 2 ^ n) (hk : 0 < k) (hkn : k < n)
  (hcop : Nat.Coprime k n) (hkk' : k * k' % n = 1) (hexp : 2 ^ k + 1 < 2 ^ n - 1)
  (εh : ε = 0 ∨ ε = 1) :
  Function.Bijective (qeps n k k' ε) ↔ ε( = 1 ↔ Even k')
```

The internal logic of the reduction (1) $\Rightarrow$ (2) is the following computation, which is exactly what `ell_of_eq` encodes:

$$\begin{array}{ccc}
(1) \ S(x) + \varepsilon = c x^{2^k+1} & \xrightarrow{P^{2^k}=S} & P = c x^{2^k+1} + x^2 + x + \varepsilon \\
\downarrow P^{2^k} + P = x^2 + x & & \downarrow (\cdot)^{2^k} \\
S = P^{2^k} & \xrightarrow{\varepsilon^{2^k} = \varepsilon} & c^{2^k} x^{2^{2k}} + x^{2^k} + c x + 1 = 0 \quad (2)
\end{array}$$

Figure 4: The telescoping computation behind `ell_of_eq`: substitute $P = (x^2 + x) + S$, raise to the 2^k power, divide by x^{2^k} .

7 Layer 2c — Theorem8Trace: the two trace identities

The $\alpha = 1$ branch needs two facts about the absolute trace, isolated here.

```

/-- Frobenius-invariance: `Tr(x^{2^k}) = Tr(x)` . -/
theorem trace_frob_shift (hn : Fintype.card F = 2 ^ n) (k : ℕ) (x : F) :
  truncTrace n (x ^ (2 ^ k)) = truncTrace n x

/-- ArtinSchreier vanishing: `Tr(t^{2^k} + t) = 0` . -/
theorem trace_artin_schreier_zero (hn : Fintype.card F = 2 ^ n) (k : ℕ) (t : F) :
  truncTrace n (t ^ (2 ^ k) + t) = 0

```

$$\begin{array}{ccc}
t & \xrightarrow{x \mapsto x^{2^k} + x} & t^{2^k} + t \\
& \searrow \text{Tr}(\cdot)=0 & \downarrow \text{Tr} \\
& & 0
\end{array}$$

Figure 5: `trace_artin_schreier_zero` as the composite $\text{Tr} \circ (x \mapsto x^{2^k} + x) = 0$, using $\text{Tr}(x^{2^k}) = \text{Tr}(x)$ (`trace_frob_shift`) and additivity.

8 Layer 3a — Theorem8C1: $\text{Tr}(x) \in \{0, 1\}$ and `gmap`

Building on `Theorem5` and the prerequisites, this file proves the trace is a *bit* and defines the trace-version Kasami map.

```

/-- `Tr(x)^2 = Tr(x)` . -/
theorem trace_sq (hn : Fintype.card F = 2 ^ n) (x : F) :
  (truncTrace n x) ^ 2 = truncTrace n x
/-- `Tr(x) ∈ {0,1}` . -/
theorem trace_bit (hn : Fintype.card F = 2 ^ n) (x : F) :
  truncTrace n x = 0 ∨ truncTrace n x = 1
/-- The trace version `g(x) = q^{Tr x}(x)` . -/
noncomputable def gmap (n k kk : ℕ) (x : F) : F := qeps n k kk (truncTrace n x) x

```

`trace_sq` is an immediate consequence of the telescoping identity together with Fermat ($x^{2^n} = x$): $\text{Tr}(x)^2 + \text{Tr}(x) = x^{2^n} + x = 0$, whence $\text{Tr}(x) \in \{0, 1\}$ because $t^2 = t$ has only the roots 0, 1 in a field.

9 Layer 3b — Q1General: the $\alpha = 1$ criterion

The trace-version analogue of `theorem_5`, i.e. the $\alpha = 1$ backbone:

```

/-- The α = 1 criterion: `g = gmap` is a permutation iff `kk + n` is odd. -/

```

```

theorem gmap_bijective_iff (hn : Fintype.card F = 2 ^ n) (hk : 0 < k) (hkn : k < n)
  (hcop : Nat.Coprime k n) (hkk' : k * kk % n = 1) (hexp : 2 ^ k + 1 < 2 ^ n - 1)
  :
  Function.Bijective (Thm8C1.gmap (F := F) n k kk) ↔ Odd (kk + n)

```

10 Layer 4 — **Equation1**: the headline thread

The top of the DAG glues everything together. Three *bridge* lemmas identify the paper's q_α (**qKasami**) with the two library backbones, after which Theorem 1 is a two-case dispatch.

10.1 Bridges: **qKasami** ↔ **qeps** / **gmap**

```

lemma qKasami_zero_eq_qeps (z : L) :
  qKasami (L := L) n k k' 0 z = Dobbertin.Thm5.qeps n k k' (0 : L) z
lemma qKasami_one_eq_gmap (z : L) :
  qKasami (L := L) n k k' 1 z = Dobbertin.Thm8C1.gmap n k k' z
lemma qKasami_mul_unit (hn : Fintype.card L = 2 ^ n) (hexp : 2 ^ k + 1 ≤ 2 ^ n - 1)
  α( : ℕ) (hx : x ≠ 0) :
  qKasami (L := L) n k k' α x * x ^ (2 ^ k + 1)
  = ∑( i ∈ Finset.Icc 1 k', x ^ (2 ^ (i * k))) + α( : L) * Tr n x

```

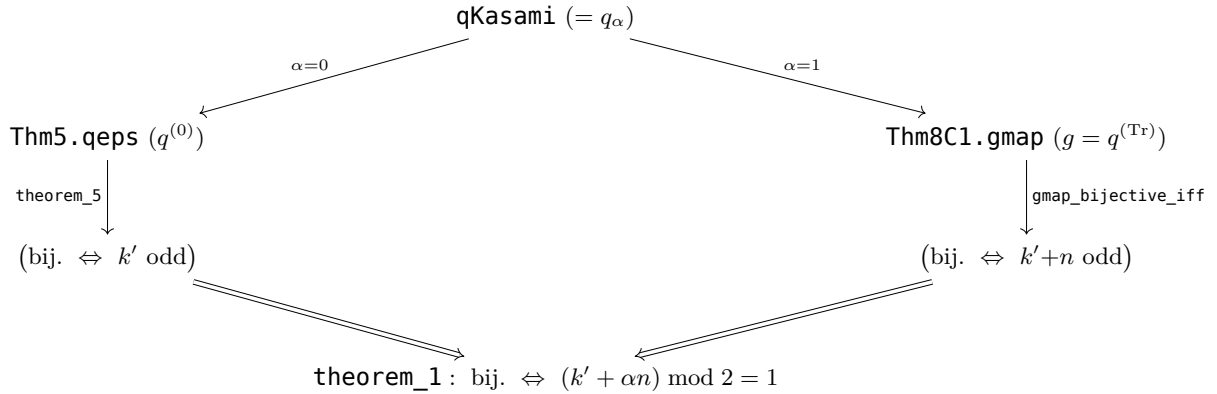


Figure 6: Assembly of **theorem_1**: the bridges (**qKasami_zero_eq_qeps**, **qKasami_one_eq_gmap**) reduce the two parities to **theorem_5** ($\alpha = 0$) and **gmap_bijective_iff** ($\alpha = 1$).

10.2 Theorem 1

Dobbertin (1999) — p. 135–136, statement of Theorem 1

Theorem 1. A generalized Kasami polynomial q_α is a permutation polynomial on L if and only if $k' + \alpha n \equiv 1 \pmod{2}$. That is, q_0 is a permutation polynomial if and only if k' is odd; q_1 is a permutation polynomial if and only if $n \equiv k' + 1 \pmod{2}$.

```

theorem theorem_1 (hn : Fintype.card L = 2 ^ n) (hk : k < n) (hcop : Nat.Coprime k n)
  (hk' : k * k' % n = 1 % n) (hk0 : 0 < k) (hexp : 2 ^ k + 1 < 2 ^ n - 1)
  α( : ℕ) (αh : α = 0 ∨ α = 1) :
  Function.Bijective (qKasami (L := L) n k k' α) ↔ (k' + α * n) % 2 = 1

```


10.3 The step (1) $\Rightarrow$ (2)

This is the first substantive step of the proof.

Dobbertin (1999) — p. 135, the passage to (2)

*Adding the 2^k -th power of this equation to itself we get ... and consequently $\ell(x) = c^{2^k} x^{2^{2k}} + x^{2^k} + cx + 1 = 0$.
(2)*

```
theorem eqn2_of_eqn1 (hn : Fintype.card L = 2 ^ n) (hkk1 : k * k' % n = 1)
  α( : ℕ) (c x : L) (hx : x ≠ 0) (h : eqn1 (L := L) n k k' α c x) :
  ell (L := L) k c x = 0
```

The proof feeds the numerator equation into `Thm5.ell_of_eq`; the side condition “ $\alpha \text{Tr}(x) \in \{0, 1\}$ ” needed by the trace-free lemma is exactly `Thm8C1.trace_bit`. See Correction 1 for why the hypothesis `hx : x ≠ 0` is present.

10.4 The two cases

Dobbertin (1999) — p. 136, the trichotomy and Case 1

Note that $\ell(x) = 0$ if and only if $cx^{2^k+1} + \sum_{i=1}^{k'} x^{2^{i_k}} + \alpha \text{Tr}(x) = 0$ or 1. As we shall see, ℓ has either one, two or four roots in L . It will be shown that at most one of them is a solution of (1).

Case 1. $c \neq \gamma^{2^k+1} + \gamma$ for all $\gamma \in L$. In this case, the homogeneous part $\ell_0(x) = \ell(x) + 1$ of (2) has no non-zero solution, since $\ell_0(x) = \frac{1}{c}(\gamma_0(x)^{2^k+1} + \gamma_0(x) + c)^2 x$ for $\gamma_0(x) = (cx^{2^k-1})^{2^{n-1}}$. Hence (2) has precisely one solution, and we are done.

```
theorem theorem_1_case1 (hn : Fintype.card L = 2 ^ n) (hk0 : 0 < k) (hkn : k < n)
  (c : L) (hc : ∀ γ : L, c ≠ γ ^ (2 ^ k + 1) + γ) :
  {x : L | ell (L := L) k c x = 0}.ncard = 1
```

Dobbertin (1999) — p. 136, Case 2

Case 2. $c = \gamma^{2^k+1} + \gamma$ for some $\gamma \in L$. Setting $Q(x) = cx^{2^k} + \gamma^2 x + \gamma$, $f = \gamma^{2^k-1} + 1/\gamma$ we have $\ell(x) = Q(x)^{2^k} + f Q(x)$ and therefore $\ell(x) = 0$ if and only if $Q(x) = 0$ or $Q(x) = 1/f \dots$

```
theorem ell_eq_Q (k : ℕ) (c γ x : L) (γh : γ ≠ 0) (hc : c = γ ^ (2 ^ k + 1) + γ) :
  ell (L := L) k c x
    = Qmap (L := L) k c γ x ^ (2 ^ k) + γ( ^ (2 ^ k - 1) + γ-1) * Qmap (L := L) k
  c γ x
```

```
theorem theorem_1_case2 (hn : Fintype.card L = 2 ^ n) (hk : k < n)
  (hcop : Nat.Coprime k n) (hkk' : k * k' % n = 1 % n) (hk0 : 0 < k)
  (hexp : 2 ^ k + 1 < 2 ^ n - 1) α( : ℕ) (αh : α = 0 ∨ α = 1)
  (hpar : (k' + α * n) % 2 = 1) (c γ : L) (hc0 : c ≠ 0) (hc : c = γ ^ (2 ^ k + 1)
  + γ) :
  {x : L | x ≠ 0 ∧ eqn1 (L := L) n k k' α c x}.ncard = 1
```

The whole case analysis is the tree of Figure 7.

Finally, `theorem_1_case2` is assembled from surjectivity of q_α (via `theorem_1`) and the denominator-clearing bridge `qKasami_mul_unit`, packaged through `Set.ncard_eq_one`:

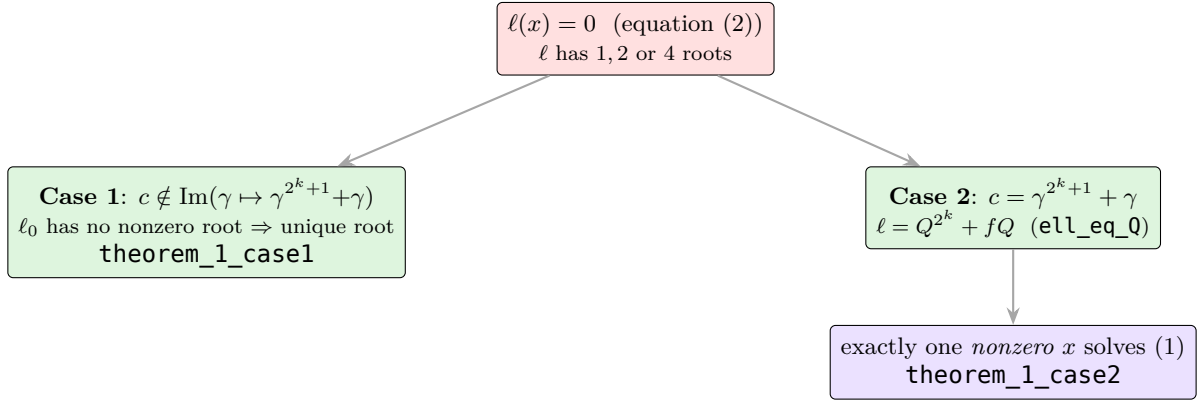


Figure 7: The case analysis of Theorem 1’s “if” direction: equation (1) has at most one solution for each fixed c .

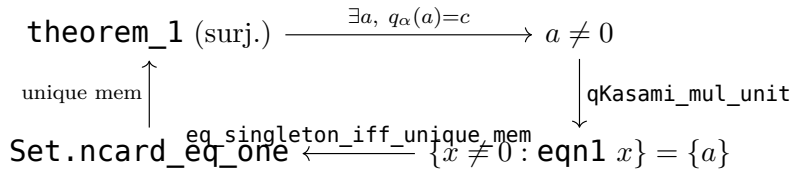


Figure 8: Assembly of `theorem_1_case2`.

11 The two documented corrections

Two internal statements are minimally strengthened relative to a literal transcription, because the library works with the polynomial-cleared form `eqn1` rather than the rational identity $q_\alpha(x) = c$. Both are documented in-source.

Correction 1 (eqn2_of_eqn1). At $x = 0$ the *cleared* equation (1) reads $0 = 0$ and is vacuously true, yet $\ell(0) = 1 \neq 0$. So “(1) $\Rightarrow \ell = 0$ ” requires $x \neq 0$. This is an artefact of clearing denominators; the paper argues about $q_\alpha(x) = c$, where the $x = 0$ case is handled separately by $q_\alpha(0) = 0$.

Correction 2 (theorem_1_case2). $x = 0$ always satisfies the cleared `eqn1`, so counting *all* solutions never gives 1. The paper’s “exactly one of the roots solves (1)” is faithfully the count of *nonzero* solutions (requiring $c \neq 0$, i.e. $\gamma \neq 1$).

12 Alternative foundations and proof paths (summary)

For completeness, the accompanying `ARCHITECTURE_AND_ALTERNATIVES.md` records alternative starting points. In brief:

- **Field object:** abstract `[Field L][Fintype L][CharP L 2]` with `card L = 2^n` (current) vs. Mathlib’s `GaloisField 2 n` (removes the cardinality-hypothesis threading).
- **Trace:** the bespoke `truncTrace` (current, ring-friendly) vs. Mathlib’s `Algebra.trace` (connects to the general theory, at the cost of an identification proof).
- **Permutation criterion:** explicit root-counting (current, mirrors the paper) vs. the $\mathbb{F}_2$ -linear-kernel / rank-nullity view (`LinearMap.injective_iff_surjective`) vs. `Polynomial` degree bounds.
- **Step (1) $\Rightarrow$ (2):** the `Thm5.ell_of_eq` telescoping (current) vs. a direct Frobenius-additive ring/linear_combination computation.